Chapter 1

Q-Learning and Blackjack

In the previous chapter we sketched the general framework of reinforcement learning — agent, environment, state, action, reward, value — and saw a tic-tac-toe example that learned to play by repeatedly updating estimates of state values. In this chapter we sharpen that idea into a specific algorithm, *Q-learning*, and use it to train an agent to play the simple casino game of blackjack. By the end of this chapter you will have written code that begins from no knowledge at all and, after a few hundred thousand simulated hands, plays blackjack about as well as it can be played.

1.1 Action-Value Functions

The tic-tac-toe player in the previous chapter learned a *state-value* function $V(s)$ — one number per state, estimating how good it is to be in that state. This worked because, in tic-tac-toe, we have a model of the game: from any state we can simulate every legal move and look at the resulting next state. We picked the move that led to the next state with the highest value.

In blackjack we don't have such a clean model. The agent does not know how the dealer will play, and the cards drawn are random. Rather than estimate the value of states, we estimate the value of *state-action pairs*. Define

$Q(s, a) =$ the expected long-run reward of taking action a in state s and acting optimally thereafter.

The function $Q(s, a)$ is called the *action-value function*. If we know Q, choosing the best action in a given state is trivial: just take the action with the largest Q-value.

$$\pi(s) = \arg\max_a Q(s, a).$$

The challenge, of course, is that we don't know Q to start with. Q-learning is a way to estimate Q from experience.

1.2 The Q-Learning Update Rule

Q-learning is an iterative algorithm. The agent interacts with the environment one step at a time. On each step, it is in state S, picks an action A, observes a reward R and a next state S', and updates its current estimate of $Q(S, A)$ to be a little closer to a new *target*. The target combines the immediate reward with the agent's current estimate of the value of the next state:

$$Q(S, A) \leftarrow Q(S, A) + \alpha\Big[R + \gamma \max_a Q(S', a) - Q(S, A)\Big].$$

Three parameters appear in this equation:

- $\alpha \in (0, 1]$ is the *learning rate*, also known as the step-size parameter. It controls how aggressively each update changes the estimate. Small α means slow but stable learning; large α means fast but noisier learning.
- $\gamma \in [0, 1]$ is the *discount factor*. It controls how much the agent cares about future rewards versus the immediate reward. $\gamma = 0$ makes the agent purely myopic (only the current reward matters); $\gamma = 1$ weights all future rewards equally. For episodic tasks like blackjack, where each hand ends after a few moves, $\gamma = 1$ is a reasonable choice.
- The action selection is governed by an *exploration parameter* ε: with probability ε, pick a random action; otherwise pick $\arg\max_a Q(S, a)$.

The update rule has a nice interpretation. The quantity in brackets is the *temporal-difference error*:

$$\delta = R + \gamma \max_a Q(S', a) - Q(S, A).$$

It is the difference between the agent's revised estimate of the value of (S, A) — "actually, the reward was R and from S' I think I can do at most $\max_a Q(S', a)$ from here" — and the agent's previous estimate. The Q-learning update moves the previous estimate a fraction α of the way toward the new target. Repeat enough times, with enough exploration, and the estimates converge.

1.3 Q-Learning in Procedural Form

Putting the pieces together, here is the algorithm in pseudocode (Sutton and Barto, Section 6.5):

Initialize $Q(s, a)$ for all s, a (often to zero)
for each episode:
 Initialize state S
 for each step of episode:
  Choose action A from S using ε-greedy on Q

Take action A, observe reward R and next state S'
$Q(S,A) \leftarrow Q(S,A) + \alpha[\,R + \gamma \max_a Q(S',a) - Q(S,A)\,]$
$S \leftarrow S'$
until S is terminal

An *episode* is one self-contained run of the task: in blackjack, one hand from initial deal to bust or stand. Q-learning trains by playing episode after episode, updating Q after every action.

1.4 The Blackjack Environment

To experiment with Q-learning we need a blackjack *environment* — a piece of software that, given the agent's action, simulates the game's response and returns a reward and the new state. We use the simplified blackjack environment from `matlab_gymnasium`[1], modeled after the OpenAI Gym Python library. The environment exposes three methods that the agent calls:

Start a new hand. Returns the initial state.

Take an action. Returns the next state, the reward, and a flag indicating whether the hand is over.

List of legal actions in the current state (typically `[0 1]` for "stand" and "hit").

The *state* in this environment is a triple `[player_sum, dealer_card, usable_ace]`: the sum of the player's cards, the value of the dealer's visible upcard, and a flag for whether the player has an ace counted as 11. The *actions* are 0 (stand — stop drawing cards) and 1 (hit — draw another card). The *reward* is $+1$ for winning the hand, -1 for losing, and 0 for a tie. Reward is delivered only at the end of the hand.

The state space is finite: there are roughly 200 reachable triples. Small enough that we can store $Q(s,a)$ as a table indexed by state, with one row per state and one column per action. In MATLAB, the natural way to do this is with the `dictionary` type introduced in Chapter 7, which lets us index by composite keys (vectors of integers, in this case) without having to flatten them to a single integer first.

1.5 Implementation Walkthrough

A Q-learning training loop in MATLAB looks roughly like this:

```
% Hyperparameters
alpha = 0.1;       % learning rate
gamma = 1.0;       % discount factor (no discount for episodic blackjack)
epsilon = 0.1;     % exploration probability
n_episodes = 100000;
```

[1] `https://github.com/bsb808/matlab_gymnasium`

```
% Initialize Q-table as a dictionary
Q = dictionary();

% Construct the environment
env = BlackJackEnv();

for ep = 1:n_episodes
    [state, ~] = env.reset();
    state_key = {state};
    if ~isKey(Q, state_key);  Q(state_key) = {zeros(1, 2)};  end

    terminated = false;
    while ~terminated
        % epsilon-greedy action selection
        if rand() < epsilon
            action = randi([0, 1]);
        else
            qvals = Q({state});
            qvals = qvals{1};
            [~, idx] = max(qvals);
            action = idx - 1;     % MATLAB indexing offset
        end

        % Take the action and observe the result
        [next_state, reward, terminated, ~, ~] = env.step(action);
        next_key = {next_state};
        if ~isKey(Q, next_key);  Q(next_key) = {zeros(1, 2)};  end

        % Q-learning update
        qcurr = Q({state});         qcurr = qcurr{1};
        qnext = Q({next_state});    qnext = qnext{1};
        target = reward + gamma * max(qnext);
        qcurr(action + 1) = qcurr(action + 1) + alpha * (target - qcurr(action + 1));
        Q({state}) = {qcurr};

        state = next_state;
    end
end
```

Read this slowly — every line is doing something the algorithm box requires. The dictionary Q is indexed by state vectors wrapped in cell arrays (a quirk of MATLAB's `dictionary` interface for non-scalar keys). Each entry stores a 2-element vector of action values, one per action. When we encounter a new state, we initialize its entry to zeros.

After enough episodes, the entries of Q stabilize. We can then *evaluate* the learned policy

by running new episodes *without* exploration ($\varepsilon = 0$) and counting wins, losses, and ties.

1.6 Why It Works

Q-learning has a remarkable property: it converges to the optimal action-value function Q^* regardless of the policy that is generating the experience, as long as every state-action pair continues to be visited and the learning rate decays appropriately. This is what is meant by saying Q-learning is *off-policy*: the policy used to choose actions during training (the ε-greedy policy) can be different from the policy whose value is being learned (the greedy policy with respect to Q).

The exploration parameter ε determines how often actions are taken that are *not* currently best. Without exploration, the agent might never discover a state-action pair that is in fact optimal. Too much exploration, and the agent wastes its time on actions it already knows are bad. In practice, schedules that start with a relatively large ε (say, 0.5) and decay it toward zero over training often work well.

1.7 Exercise: A Different Reinforcement Learning Problem

A worthwhile capstone is to step away from blackjack and apply Q-learning to a problem of your own choosing. Some examples:

```
[state, ~] = env.reset();   % integer state, with as many as you need
[s, r, terminated, ~, ~] = env.step(action);   % action in 1..nActions
```

- A simplified inventory-management problem: each day, decide whether to reorder, given uncertain demand.
- A toy elevator dispatcher: given a current floor and a set of pending requests, choose which to service next.
- A multi-armed bandit, the simplest possible RL problem (single state, multiple actions, stochastic rewards) — a useful sanity check before tackling anything more complicated.

In every case the steps are the same: define the state, action, and reward; build an environment that implements `reset()` and `step(action)`; pick reasonable values of α, γ, and ε; train; evaluate the learned policy.

1.8 Chapter Review

Q-learning is a model-free reinforcement learning algorithm that estimates the action-value function $Q(s, a)$ — the expected long-run reward of taking action a in state s — by iterative temporal-difference updates. The learning rate α controls update step size; the discount factor γ trades off immediate vs. long-run reward; the exploration parameter ε keeps the agent from getting stuck on apparently-good actions before it has tried other options.

For a finite, small state space we can store Q as a table. In MATLAB the `dictionary` type lets us index a Q-table by compound state keys without manual flattening.

The blackjack environment in `matlab_gymnasium` is a clean small example: discrete states, two actions, rewards only at episode end. After enough training episodes, Q-learning converges to a policy close to the known-optimal blackjack strategy.

1.9 Exercises

Exercise 1.1. Train a Q-learning agent on blackjack for 10 000, 100 000, and 1 000 000 episodes. Evaluate each trained agent over 10 000 test episodes (with $\varepsilon = 0$). Plot the average reward per episode against the training duration. At what point do further episodes stop helping?

Exercise 1.2. Train a Q-learning agent with $\varepsilon = 1$ (i.e., always take a random action during training). Evaluate the resulting greedy policy. Then train with $\varepsilon = 0$ (always greedy during training). Compare and explain the resulting performance.

Exercise 1.3. Design a small reinforcement learning problem of your own (see Section 1.7 for ideas). Write down the state, action, and reward definitions; implement an environment with `reset()` and `step(action)` methods; train a Q-learning agent on it; and evaluate the learned policy. Submit a short write-up describing the problem, your design choices, and the results.